GPS Guardián Escolar:
análisis, diseño e implementación de una plataforma de acompañamiento de rutas escolares con seguimiento de ubicación

Nicolas Estid Ruiz Sastoque

Jhon Camilo Suaza Sánchez

Kevin Andrey Culma Gómez

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software · Ficha 3145555

Neiva, Huila, 5 de noviembre de 2026

Resumen

GPS Guardián Escolar es un proyecto formativo de análisis y desarrollo de software orientado al acompañamiento de trayectos escolares mediante la asociación entre estudiantes, acudientes, dispositivos y rutas. Este artículo examina el problema que motivó su desarrollo, las decisiones de arquitectura, los principales flujos funcionales y el alcance de las evidencias técnicas conservadas hasta el 5 de noviembre de 2026. Se empleó un estudio de caso tecnológico, de carácter descriptivo y aplicado, sustentado en la especificación de requisitos, los documentos de diseño, los repositorios del proyecto y los registros disponibles del despliegue. El sistema implementa una aplicación cliente con Expo y React Native, una API con Spring Boot, persistencia PostgreSQL administrada mediante Liquibase y comunicación REST y WebSocket para el intercambio de información de ubicación. El análisis identifica mecanismos de autenticación, vinculación de dispositivos, gestión de rutas, seguimiento de trayectos, zonas seguras y notificaciones, así como restricciones que afectan su uso. Como hallazgo central, el código permite describir una solución integrada y sus controles, pero la evidencia revisada no constituye una medición concluyente de disponibilidad, precisión geográfica, autonomía de batería o entrega de notificaciones en condiciones reales. Se propone una evaluación gradual que combine pruebas técnicas, validación con dispositivos físicos, protección de datos de menores y criterios verificables de aceptación.

Palabras clave: seguimiento escolar, geolocalización, React Native, Spring Boot, seguridad de la información, arquitectura de software.

Abstract

GPS Guardián Escolar is a software development training project designed to support school journeys through the relationship among students, guardians, devices, and routes. This paper examines the problem that motivated the system, its architectural decisions, core functional flows, and the scope of technical evidence available as of November 5, 2026. A descriptive, applied technological case-study approach was used, drawing on requirements, design documents, project repositories, and existing deployment records. The system uses an Expo and React Native client, a Spring Boot API, PostgreSQL persistence managed by Liquibase, and REST and WebSocket communication for location updates. The review describes authentication, device linking, route management, trip tracking, safe zones, notifications, and their practical limitations. The primary finding is that the code provides evidence of an integrated technical solution but does not yet establish real-world availability, location accuracy, battery performance, or successful notification delivery. A staged evaluation is proposed to combine technical tests, physical-device validation, protection of children's personal data, and auditable acceptance criteria.

Keywords: school transportation, location tracking, React Native, Spring Boot, information security, software architecture.

Introducción

Un recorrido escolar involucra decisiones cotidianas que, aunque suelen parecer sencillas, generan incertidumbre: saber cuándo salió el estudiante, reconocer si el desplazamiento transcurre por la ruta esperada y disponer de información oportuna cuando la ubicación deja de actualizarse. En el transporte escolar, esa incertidumbre afecta tanto a las familias como a quienes acompañan la operación. La tecnología puede aportar información útil, pero solo si la ubicación se vincula con una identidad autorizada, un trayecto concreto y reglas comprensibles para las personas que utilizan el sistema.

GPS Guardián Escolar surgió en el contexto del programa Análisis y Desarrollo de Software del Servicio Nacional de Aprendizaje (SENA), ficha 3145555. El equipo se propuso construir una aplicación capaz de relacionar los registros de estudiantes con rutas escolares y teléfonos autorizados, conservar información sobre los trayectos y poner a disposición del acudiente una vista de seguimiento. El énfasis no se limita a dibujar un marcador sobre un mapa; incluye responsabilidades de registro, control de acceso, envío de coordenadas, notificaciones y persistencia coherente de los datos (GPS Guardián Escolar, 2026a, 2026b).

La construcción del proyecto atravesó varias etapas. La especificación inicial concentró necesidades, actores y expectativas; posteriormente, el desarrollo se organizó en repositorios separados para backend, frontend, base de datos, infraestructura y documentación. Esa organización permitió distribuir el trabajo, aunque también hizo más exigente la coordinación de interfaces y versiones. Durante octubre y los primeros días de noviembre de 2026 se documentaron cambios relacionados con autenticación, vinculación QR, rutas preferidas, seguimiento de ubicación, migraciones y despliegue. Por ello, una descripción académica

responsable necesita distinguir el diseño planteado, el comportamiento visible en el código y aquello que fue realmente probado.

Este artículo presenta el sistema como un estudio de caso de ingeniería de software aplicada. Su objetivo no es atribuir al producto una eficacia que todavía no se ha medido, sino explicar cómo se configuró una solución tecnológica concreta, cuáles fueron las decisiones más relevantes y qué resultados pueden sostenerse con la evidencia disponible. La diferencia entre una función implementada y una función validada es particularmente importante cuando intervienen dispositivos móviles, servicios externos y datos de niños, niñas y adolescentes.

Planteamiento del problema

La comunicación durante un trayecto escolar suele depender de llamadas, mensajes dispersos o información transmitida después de ocurrido un cambio. Estos mecanismos pueden resultar suficientes en situaciones ordinarias, pero ofrecen poca trazabilidad cuando un acudiente necesita distinguir una demora normal de una interrupción de la ubicación. La dificultad no se reduce a conocer la posición: es necesario saber a qué estudiante corresponde, quién puede consultarla y si el registro está actualizado.

En un escenario de seguimiento, una coordenada aislada puede inducir a error. Un punto geográfico puede pertenecer a un trayecto ya finalizado; también puede reflejar la última información conocida de un teléfono sin conexión. Mostrarla sin contexto puede transmitir una certeza que el sistema no posee. Por eso GPS Guardián Escolar incorpora el concepto de trayecto, estados de operación y una revisión de antigüedad de la última ubicación. El código del backend contempla un umbral predeterminado de 120 segundos y una comprobación programada aproximadamente cada 30 segundos para detectar información desactualizada durante trayectos activos (GPS Guardián Escolar, 2026a).

Otra dificultad consiste en establecer la relación correcta entre el estudiante y el dispositivo que genera las coordenadas. Un teléfono no puede convertirse, por el simple hecho de conocer un enlace, en una fuente de información confiable para cualquier perfil escolar. La aplicación necesita procesos de vinculación, autorización y revocación. La presencia de un código QR facilita la operación, pero también introduce riesgos si el contenido se reenvía, se captura o se utiliza fuera del contexto previsto. La funcionalidad de vinculación, por tanto, pertenece tanto al problema de usabilidad como al de seguridad.

El proyecto también enfrenta las restricciones de las plataformas. Los navegadores web no ofrecen las mismas garantías de ejecución prolongada que una aplicación Android instalada. Incluso en Android, los permisos de ubicación y las medidas de ahorro energético pueden detener tareas que se esperaba mantener activas. La documentación de Expo aclara que las tareas de localización en segundo plano requieren compilaciones de desarrollo o nativas y no pueden validarse de manera equivalente en Expo Go (Expo, s. f.-a). Esto impide equiparar una demostración web satisfactoria con una comprobación completa del seguimiento móvil.

El problema que guía el trabajo puede formularse así: ¿cómo integrar una solución multiplataforma que permita registrar estudiantes y rutas, vincular de manera controlada los dispositivos y consultar el estado de los trayectos, preservando al mismo tiempo la trazabilidad de los datos y delimitando las garantías reales de funcionamiento? Esta pregunta se responde mediante el análisis de la arquitectura construida, no mediante una comparación estadística con servicios comerciales que el equipo no evaluó directamente.

Justificación y pertinencia

El valor formativo del proyecto consiste en relacionar necesidades identificables con decisiones verificables de diseño. Un sistema de ubicación escolar exige modelado de datos, APIs, autenticación, comunicación en tiempo real, tratamiento de errores y criterios de operación. Ese conjunto crea un escenario pertinente para el programa ADSO porque obliga a coordinar análisis, desarrollo, pruebas, documentación y despliegue en vez de abordar cada tema como ejercicio aislado.

Desde la perspectiva del usuario, un registro de trayecto asociado al estudiante y a su ruta puede ordenar la información y facilitar la consulta posterior. El historial puede permitir revisar eventos sin depender de mensajes informales. No obstante, se trata de beneficios potenciales derivados del diseño: hasta disponer de pruebas con usuarios y registros de uso, no corresponde afirmar que la aplicación haya reducido incidentes, tiempos de respuesta o niveles de preocupación de los acudientes. Esta distinción conserva la credibilidad de los resultados.

En el plano tecnológico, la separación entre cliente, servicios y datos aporta una estructura que permite trabajar en cambios específicos. La interfaz puede evolucionar sin modificar las tablas directamente; el backend concentra reglas de permisos y persistencia; las migraciones SQL quedan versionadas por fuera del ciclo del cliente. La decisión tiene costos, como la necesidad de mantener contratos y entornos sincronizados, pero esos costos son visibles y pueden administrarse con una disciplina de integración.

Finalmente, la naturaleza de la información obliga a considerar la responsabilidad ética del diseño. La ubicación y los datos escolares son información personal que requiere una finalidad definida, acceso limitado y políticas de conservación. La Ley 1581 de 2012 incluye reglas especiales para datos de niños, niñas y adolescentes, y su reglamentación desarrolla

condiciones para el tratamiento en situaciones permitidas (Congreso de Colombia, 2012; Presidencia de la República de Colombia, 2013). El proyecto no puede presentarse como listo para operación institucional únicamente porque sus endpoints respondan; también necesita decisiones documentadas sobre consentimiento, transparencia y atención de solicitudes de titulares.

Objetivos del estudio

El objetivo general es analizar y documentar el desarrollo de GPS Guardián Escolar como plataforma de acompañamiento de rutas escolares, describiendo su arquitectura, los flujos funcionales implementados, las medidas de protección de la información y los límites de su validación técnica al 5 de noviembre de 2026.

De ese objetivo se desprenden cinco propósitos específicos. Primero, caracterizar a los actores y los casos de uso que articulan registro, seguimiento y consulta. Segundo, describir el papel del frontend, el backend, la persistencia y los servicios externos. Tercero, examinar decisiones de autenticación, vinculación y autorización. Cuarto, identificar evidencias que respalden el estado de implementación sin confundirlas con resultados experimentales. Quinto, formular un plan reproducible para completar la evaluación funcional, de seguridad y de operación.

El alcance del artículo se limita a los repositorios, los documentos institucionales adjuntos y las evidencias disponibles del proyecto. No constituye auditoría jurídica, estudio de movilidad escolar, ensayo clínico, estudio de impacto social ni certificación de un servicio en producción. Tampoco se utilizaron datos personales reales de estudiantes para construir resultados. Donde el expediente no ofrece mediciones, la exposición emplea términos como «previsto», «implementado en el código» o «pendiente de comprobación», según corresponda.

Antecedentes técnicos y conceptos de referencia

La localización por dispositivos móviles ha hecho posible diseñar aplicaciones que combinan registros temporales con representaciones cartográficas. Sin embargo, el GPS por sí mismo no resuelve la identidad de la fuente ni el uso legítimo de la información. La coordenada solo adquiere sentido cuando se relaciona con una entidad, un período y un permiso de acceso. GPS Guardián Escolar traduce esa necesidad a estudiantes, rutas, trayectos y dispositivos vinculados. La estructura se observa tanto en el modelo de datos como en la API que administra esas relaciones (GPS Guardián Escolar, 2026a, 2026c).

El intercambio por HTTP responde adecuadamente a operaciones discretas: registrar un estudiante, crear una ruta, consultar historial y modificar preferencias. En cambio, el seguimiento de coordenadas sucesivas se beneficia de un canal de publicación y suscripción que reduzca la necesidad de consultas continuas. El proyecto utiliza REST para solicitudes y STOMP sobre WebSocket para difundir actualizaciones relacionadas con un trayecto. Este diseño no elimina retrasos de red ni equivale a transmisión garantizada; proporciona una vía adecuada para distribuir eventos a clientes conectados (Spring, s. f.-a).

La calidad del producto no debe reducirse a «funciona» o «no funciona». ISO/IEC 25010:2023 proporciona un modelo para examinar dimensiones de calidad del software; en este caso resultan especialmente pertinentes adecuación funcional, confiabilidad, seguridad, mantenibilidad, desempeño y capacidad de interacción. La referencia se utiliza como marco conceptual y no como afirmación de certificación (International Organization for Standardization [ISO], 2023). Esta perspectiva ayuda a evitar que una prueba de inicio de sesión sea tratada como sustituto de una prueba de continuidad del GPS.

En seguridad, las guías de OWASP y el marco de desarrollo seguro del NIST recomiendan pensar en los riesgos desde el diseño, documentar mecanismos verificables y revisar los resultados mediante procedimientos repetibles (National Institute of Standards and Technology [NIST], 2022; OWASP Foundation, s. f.). Para una aplicación escolar, esa orientación conduce a preguntas prácticas: qué cuenta puede consultar qué estudiante, cómo se revoca un teléfono extraviado, cuánto tiempo se guardan las coordenadas y qué ocurre cuando un dispositivo deja de enviar información.

Metodología: estudio de caso tecnológico

El trabajo se organizó como un estudio de caso de ingeniería de software aplicada con alcance descriptivo. La unidad de análisis fue el proyecto GPS Guardián Escolar y, dentro de él, cinco conjuntos de artefactos: requisitos, interfaz de usuario, servicios del backend, estructura de datos e infraestructura de despliegue. La evidencia se obtuvo de los documentos de especificación, diseño y pruebas conservados por el equipo; del código accesible en los repositorios; y de notas técnicas sobre el despliegue. No se efectuó una encuesta ni una intervención experimental con familias para este artículo.

La lectura de artefactos siguió un orden orientado a reconstruir decisiones. Primero se identificaron actores, entidades y operaciones descritas por los requisitos. Luego se revisó la organización del frontend y de los controladores del backend para determinar qué recorridos tenían representación concreta. En un tercer momento se relacionaron esos recorridos con entidades, tablas y migraciones. Por último, se comparó la configuración de Docker, los archivos de variables de entorno y los reportes de puesta en marcha. Cuando un documento antiguo mencionaba un endpoint que no figuraba en el controlador del corte examinado, se priorizó el código para describir el comportamiento disponible y se dejó la diferencia como hallazgo.

El corte de referencia requiere especial cuidado. Los repositorios de implementación usan ramas de desarrollo, QA y publicación que pueden divergir en commits. Al 5 de noviembre se identificaron diferencias entre `dev` o `develop`, `qa` y `main` en los cuatro repositorios de código. Para no construir una descripción artificial mezclando comportamientos de momentos distintos, los hallazgos funcionales específicos de esta revisión se asignan al código visible en `main` y se aclara cuando corresponden a antecedentes de desarrollo. Un SHA fijado permite repetir la inspección; mencionar una rama sin fecha no ofrece la misma precisión.

El análisis se apoyó en tres categorías de evidencia. «Presencia en código» significa que se encontró una clase, configuración, migración o flujo correspondiente. «Evidencia operativa reportada» indica que el expediente conserva logs, capturas o notas sobre una ejecución determinada, pero sin que se haya repetido necesariamente en esta revisión. «Validación pendiente» se reserva para funciones que necesitan un ensayo controlado, un dispositivo específico, una cuenta externa o la intervención de usuarios. Se evita presentar un test existente en el repositorio como prueba aprobada sin un resultado de ejecución verificable.

El método tiene limitaciones reconocibles: la inspección estática no confirma la disponibilidad de un servicio, la documentación de despliegue refleja un momento concreto y las dependencias de Google Maps, correo y notificaciones requieren cuentas y configuraciones externas. Estas limitaciones no invalidan la descripción de la arquitectura; delimitan qué se puede concluir de ella. Se adopta esta precaución porque una evaluación tecnológica pierde utilidad si convierte suposiciones razonables en resultados observados.

Actores, necesidades y alcance funcional

El estudiante es la persona cuyo recorrido se desea acompañar; el acudiente es quien administra o consulta la información según sus derechos dentro del sistema. Una cuenta

administradora puede supervisar aspectos del servicio y un dispositivo infantil vinculado cumple una función específica en el envío de ubicación. La distinción es importante: un dispositivo no debería recibir por defecto las mismas capacidades administrativas que el adulto responsable. El backend incluye validaciones de permisos y de relación con el estudiante, y el frontend diferencia pantallas y controles según el contexto de usuario (GPS Guardián Escolar, 2026a, 2026b).

Entre las necesidades funcionales se encuentran el registro de cuentas, verificación de correo, recuperación de acceso, administración de estudiantes, asignación de rutas, vinculación mediante QR o código, consulta de trayectos, registro de coordenadas, configuración de zonas seguras y consulta de notificaciones. Estas capacidades se distribuyen en módulos del backend y servicios del cliente, en lugar de apoyarse en un único controlador general. El alcance real de cada operación debe leerse junto con las reglas de autorización que ejecuta el servidor.

El proyecto incorpora también necesidades transversales: disponer de un entorno repetible para el equipo, aplicar cambios de base de datos en orden y rastrear problemas detectados durante la integración. La separación del repositorio de base de datos responde a la necesidad de controlar el esquema con Liquibase; el repositorio de infraestructura reúne la secuencia de arranque. Esos elementos no son visibles para el acudiente, pero condicionan la calidad del producto final porque de ellos dependen la integridad de las relaciones y la disponibilidad de la aplicación.

El límite funcional debe quedar explícito: el sistema ayuda a observar información de ubicación y estado de trayecto; no sustituye protocolos escolares de emergencia, servicios de seguridad ni comunicación directa con responsables del transporte. Una alerta de posición desactualizada no demuestra por sí sola que haya ocurrido un incidente. Precisamente por esa

razón, el mensaje implementado por el backend admite causas alternativas, como falta de internet, interrupción de GPS o suspensión del uso compartido de ubicación.

Arquitectura general de la solución

La arquitectura adoptada es cliente-servidor, con organización modular en el backend. El cliente se construye con Expo, React Native y Expo Router; utiliza servicios HTTP y comunicación STOMP para consultar o recibir información. El backend está desarrollado en Java 17 con Spring Boot 3.5.16; expone controladores REST y módulos como `auth`, `students`, `routes`, `trips`, `zones`, `maps`, `notifications` y `admin`. La persistencia se apoya en PostgreSQL 15 y la administración del esquema corresponde a Liquibase (GPS Guardián Escolar, 2026a, 2026b, 2026c).

El modelo evita que la interfaz acceda directamente a las tablas. Cuando un acudiente consulta un trayecto, el cliente formula una solicitud a la API; el controlador recibe parámetros, el servicio aplica las reglas del dominio y el repositorio consulta las entidades persistidas. El servidor devuelve una representación destinada al cliente. Esta separación permite que las decisiones sobre acceso compartido, estado del trayecto y legitimidad de la vinculación permanezcan en el backend, aunque los elementos de interfaz se oculten o se muestren en función del rol.

La lógica del dominio está agrupada por responsabilidades reconocibles. El módulo de autenticación participa en la emisión y validación de credenciales; estudiantes gestiona registros, relaciones y dispositivos; rutas mantiene recorridos y paradas; trayectos procesa posiciones y eventos; notificaciones organiza avisos y preferencias; mapas actúa como intermediario de servicios externos. Esta distribución reduce el riesgo de concentrar reglas incompatibles en una

sola capa. No implica microservicios independientes: el backend constituye una aplicación Spring Boot desplegada como unidad, con módulos internos.

La comunicación presenta dos caminos distintos. En REST, cada solicitud produce una respuesta en la que el servidor puede validar identidad, permisos y contenido. En WebSocket se establece un canal de mensajes para publicar coordenadas hacia suscriptores autorizados. La configuración del proyecto utiliza `/ws` como punto de conexión STOMP, `/app` como prefijo de aplicación y `/topic` para distribución. Un canal de tiempo real mejora la oportunidad de actualización en la interfaz, pero requiere probar reconexiones, autenticación de la suscripción y comportamiento cuando el cliente permanece desconectado.

En la operación local, Docker Compose articula cuatro servicios: `database`, `liquibase`, `backend` y `frontend`. PostgreSQL atiende las consultas; Liquibase prepara las estructuras y termina después de ejecutar los cambios; el backend espera que la migración concluya; y el frontend web se publica mediante Nginx, que también puede dirigir las rutas `/api` y `/ws` al servidor. Esta secuencia representa una dependencia real: iniciar la aplicación antes de preparar el esquema puede provocar fallos de validación de JPA y errores que no se solucionan recargando la página.

La elección de herramientas favorece la continuidad entre el ambiente de desarrollo y los entornos de demostración. Sin embargo, no elimina por sí sola la gestión de secretos, el mantenimiento de versiones de imágenes ni el monitoreo. El repositorio de infraestructura ya diferencia credenciales de administración de PostgreSQL de una cuenta de aplicación con privilegios reducidos. Esa decisión es relevante para limitar el alcance de un fallo de credenciales, aunque su efectividad en un ambiente real depende de una configuración revisada y verificada.

Diseño del cliente e interacción con usuarios

El frontend organiza pantallas de autenticación y áreas de navegación principal. Las rutas de `app/(auth)` corresponden a ingreso, registro, verificación de correo y recuperación de contraseña. En `app/(tabs)` aparecen secciones como inicio, estudiantes, seguimiento, historial, zonas, notificaciones, perfil y administración. Esta agrupación proporciona un recorrido identificable a partir de las tareas del usuario y facilita ubicar el código de cada vista cuando se identifica un problema (GPS Guardián Escolar, 2026b).

La interfaz no puede evaluarse solamente por su apariencia. Una pantalla de vinculación, por ejemplo, debe ofrecer una alternativa cuando la cámara no reconoce el QR o el usuario no concede permiso. El proyecto contempla el ingreso manual de un código y ajustes para diferenciar la experiencia web de la experiencia móvil. Desde el punto de vista de interacción, la alternativa evita que un fallo de cámara cierre por completo la tarea, aunque se debe asegurar que el mecanismo manual conserve controles equivalentes de vigencia y autorización.

La representación cartográfica exige especial cuidado con los estados de carga y error. Un mapa vacío puede significar que todavía no se ha recibido una coordenada, que el proveedor externo respondió con un error, que el usuario no tiene acceso o que el trayecto terminó. Mostrar una interfaz idéntica para todos esos casos dificulta actuar correctamente. El frontend dispone de servicios separados para mapas, rutas y seguimiento, lo cual permite manejar los mensajes cerca de la función correspondiente. La revisión funcional debería comprobar que cada estado presenta información comprensible y no expone datos de otros estudiantes.

El diseño multiplataforma tiene ventajas de mantenimiento, pero no puede borrar las diferencias entre dispositivos. En navegador se usan capacidades web y una versión estática servida por Nginx; en Android intervienen permisos, tareas nativas y decisiones del fabricante

sobre ejecución en segundo plano. Expo advierte expresamente que algunas funciones de localización requieren un build nativo, de modo que una pantalla que funciona en Expo Go no basta para validar un recorrido continuo (Expo, s. f.-a). En consecuencia, el proyecto necesita criterios de aceptación separados para web y para Android instalado.

Una decisión importante es la distinción entre el acudiente y el dispositivo del estudiante. La interfaz infantil concentra acciones relacionadas con su vinculación y ubicación, mientras que las facultades de administración y consulta permanecen sujetas a permisos. Esta decisión no debe descansar únicamente en el hecho de ocultar botones. El servidor debe comprobar la identidad asociada al dispositivo y el estudiante que intenta modificar. La separación entre interfaz y autorización fortalece la coherencia del modelo cuando un usuario modifica manualmente una solicitud HTTP.

Autenticación, sesiones y recuperación de acceso

El proceso de autenticación comienza con el registro y la comprobación de identidad del usuario. El controlador de autenticación expone operaciones de registro, inicio de sesión, consulta del perfil actual, verificación de correo, confirmación de acceso, recuperación y cambio de contraseña, cierre de sesión y verificación en dos pasos. La presencia de estos endpoints muestra que el acceso fue tratado como un flujo completo, aunque el funcionamiento efectivo de cada canal de correo depende de las credenciales y del proveedor configurado (GPS Guardián Escolar, 2026a).

La API utiliza tokens JWT para identificar solicitudes. El estándar RFC 7519 define la estructura de los JSON Web Tokens y ofrece un marco útil para entender sus afirmaciones y límites; no significa que cualquier token firmado garantice un sistema seguro (Jones et al., 2015). En GPS Guardián Escolar el servidor también mantiene información de sesiones y dispositivos.

Eso permite incorporar controles complementarios a la firma del token, como el seguimiento de accesos y las decisiones de revocación, que se deben verificar con casos de prueba específicos.

La verificación del correo persigue un propósito distinto de la autorización. Confirmar una dirección ayuda a controlar el canal de comunicación, pero no otorga automáticamente acceso a los datos de cualquier estudiante. La autorización debe revisar relaciones y permisos en cada operación sensible. El proyecto modela roles y permisos en la base de datos y utiliza condiciones del backend para distinguir operaciones de lectura, escritura y administración. El caso de estudiantes compartidos exige particular atención: una persona puede tener acceso a consultar un registro sin estar facultada para modificar o retirar a otros acudientes.

El cierre de sesión y el control de nuevos dispositivos son fuentes habituales de incidentes de integración. Durante el trabajo del proyecto se registraron correcciones relacionadas con confirmación de acceso desde otro dispositivo y gestión de sesiones. Para analizarlas resulta más útil formular pruebas observables que describirlas como «seguras»: iniciar sesión en un equipo, solicitar acceso desde otro, comprobar el canal de confirmación, negar la solicitud y verificar que el segundo equipo no obtiene acceso. Las respuestas a esos casos deben registrarse con fecha, versión de backend y configuración de correo.

La recuperación de contraseña requiere separar la identidad de la cuenta de la posesión temporal de un enlace o código. Los tokens transitorios deben tener caducidad, una finalidad limitada y medidas contra repetición. El repositorio contiene utilidades para proteger códigos y sesiones mediante representaciones hash, pero la existencia de esas clases no basta para certificar el conjunto del flujo. Las guías de OWASP sobre autenticación y el SSDF del NIST sirven como referencia para diseñar una evaluación con límites de intentos, gestión de secretos y errores de respuesta (NIST, 2022; OWASP Foundation, s. f.).

El riesgo residual merece una lectura prudente. Las variables `JWT_SECRET`, los proveedores de correo y los orígenes CORS pueden convertir una implementación correcta en una instalación insegura si se configuran con valores improvisados. Por ello, la documentación técnica prescribe secretos externos al repositorio y una llave JWT de suficiente entropía. Para el artículo, esa disposición se considera parte del diseño; la verificación de su cumplimiento pertenece a la auditoría del entorno concreto.

Gestión de estudiantes, acudientes y vínculos

La entidad estudiante actúa como centro de varias relaciones: puede tener acudientes autorizados, contactos de emergencia, rutas asignadas y dispositivos vinculados. El backend dispone de operaciones para registrar, actualizar y consultar estudiantes, asignar rutas y gestionar relaciones con acudientes. La persistencia separa esas relaciones en tablas y entidades específicas, en lugar de introducir toda la información en una sola estructura de texto (GPS Guardián Escolar, 2026a, 2026c).

La relación con un acudiente no es solo un dato de contacto. Determina quién consulta la ubicación y quién puede efectuar cambios. El servicio de acceso a estudiantes valida la pertenencia y el tipo de permiso antes de operaciones relevantes. Cuando un estudiante se comparte con otro usuario, el sistema necesita distinguir la consulta del acto de administrar la relación. Esta distinción disminuye el riesgo de que una función de colaboración se convierta en una forma de acceso permanente no previsto.

Los contactos de emergencia amplían el contexto operativo, pero su presencia tampoco debe confundirse con una función automática de rescate. La aplicación permite almacenar información que podría ser útil en un procedimiento institucional; no consta en el expediente una integración con servicios de emergencia ni un protocolo probado de despacho. Al elaborar un

manual para usuarios, la diferencia es esencial para que las expectativas coincidan con el alcance real del producto.

El código conserva mecanismos de desvinculación de dispositivos asociados con estudiantes. La posibilidad de retirar una asociación es tan importante como crearla, sobre todo cuando un teléfono se extravía, se reemplaza o deja de ser utilizado por el estudiante. Una prueba completa deberá verificar que, después de desvincular, el dispositivo anterior no puede consultar un perfil protegido ni continuar publicando coordenadas usando una credencial previa. La validación debe contemplar sesiones abiertas y solicitudes concurrentes, no únicamente la desaparición de un botón.

Vinculación mediante QR y código manual

El código QR simplifica el intercambio inicial de información, pero no representa por sí mismo un permiso ilimitado. En GPS Guardián Escolar el acudiente solicita un código de vinculación y el dispositivo del estudiante utiliza la pantalla de escaneo o una alternativa de ingreso manual. El backend recibe la solicitud, verifica la información y asocia el identificador del dispositivo al estudiante permitido. Esta secuencia convierte una acción visual en una operación de identidad y acceso que debe quedar trazada (GPS Guardián Escolar, 2026a, 2026b).

El proceso presenta varios casos alternos que merecen análisis individual. Un código puede estar vencido, haber sido usado, pertenecer a otro estudiante o contener caracteres mal interpretados. La cámara puede carecer de permisos o fallar por condiciones de luz, mientras que el ingreso manual puede sufrir errores de digitación. El comportamiento esperado no es que todos los fallos terminen en un mensaje genérico; el sistema debe rechazar de forma segura la vinculación y ofrecer al usuario una salida comprensible sin exponer información sensible sobre terceros.

La introducción de confirmaciones antes de completar la vinculación responde a un riesgo operativo: un estudiante o acudiente puede interpretar que un teléfono ha quedado asociado sin comprender a qué perfil corresponde. Una pantalla de revisión ayuda a evitar equivocaciones, pero el valor real se comprueba con pruebas de interacción. Debe observarse si el usuario identifica correctamente al estudiante, entiende la solicitud de permisos y puede cancelar antes de guardar el vínculo.

El soporte de enlaces profundos y códigos QR también depende de la compilación móvil y su firma. La configuración Android puede asociar enlaces HTTPS con una aplicación concreta mediante un archivo de relación de dominio y una huella de certificado. El expediente del proyecto reconoce que, si esa relación no coincide con el binario instalado, el enlace puede abrirse en el navegador. No se dispone de una prueba concluyente para todos los dispositivos, por lo que debe describirse como una condición de instalación y no como comportamiento universal garantizado.

Rutas, paradas y selección de trayecto

El dominio de rutas incluye recorridos, paradas y asignaciones de estudiantes. Su sentido va más allá de mostrar una línea sobre un mapa: una ruta puede tener un origen, un destino, puntos intermedios y estudiantes asignados. La capa de servicios valida que el estudiante tenga la ruta solicitada antes de permitir su utilización en un trayecto. Esa validación previene que una coordenada se asocie por error con una ruta que no corresponde a la relación autorizada (GPS Guardián Escolar, 2026a).

La versión revisada incorpora una ruta preferida en la asignación activa de cada estudiante. La base de datos contiene una migración que agrega `student_routes.is_preferred` y restringe duplicados en las asignaciones activas. Cuando

se crea un trayecto sin identificador explícito de ruta, el servicio busca primero la asignación preferida activa y válida. Si no la encuentra, puede utilizar una única ruta asignada; cuando hay más de una alternativa sin preferencia, exige que el acudiente seleccione una. El criterio evita elegir silenciosamente una ruta entre varias opciones posibles.

Esta lógica muestra cómo una regla de negocio atraviesa diferentes capas. En la base existe un atributo que expresa preferencia; el servicio lo consulta y determina qué ruta corresponde; el cliente puede presentar la elección; la documentación debe explicar la condición para el usuario. Si cualquiera de estos elementos queda desactualizado, aparecen comportamientos confusos: el frontend podría asumir una ruta automática que el backend rechaza o una migración incompleta podría impedir leer el campo requerido.

La función de rutas también exige reglas para cambios posteriores. Si una ruta se desactiva o se elimina lógicamente, no debería convertirse en destino automático de un nuevo trayecto solo porque conserva una asignación antigua. La implementación revisada filtra rutas eliminadas al resolver la preferida. Este detalle es importante porque demuestra que la trazabilidad histórica y el uso operativo son conceptos distintos: conservar una entidad para auditoría no obliga a permitir nuevas operaciones con ella.

Trayectos, coordenadas y seguimiento en tiempo real

Un trayecto representa una instancia temporal de desplazamiento asociada a un estudiante y una ruta. En el backend se identifica mediante UUID, registra inicio y estado y admite coordenadas ordenadas en el tiempo. Este diseño permite separar una ruta reutilizable de cada recorrido concreto. Dos trayectos pueden pertenecer a una misma ruta sin confundirse, y un historial puede consultar operaciones anteriores sin alterar el seguimiento activo (GPS Guardián Escolar, 2026a).

El controlador de trayectos disponible en el corte `main` revisado permite consultar el trayecto activo de un estudiante, listar trayectos con paginación, crear un registro, eliminarlo lógicamente y registrar o consultar coordenadas. El servicio impide crear un nuevo trayecto cuando ya existe uno activo para el estudiante y valida la asignación de ruta. La creación registra inicialmente `IN_PROGRESS` y genera el evento de inicio. Es importante no reproducir mecánicamente contratos de documentación antiguos: en el controlador examinado no aparece un endpoint `PATCH /api/trips/{id}/status`, aunque documentos de etapas anteriores lo mencionaran.

El envío de coordenadas exige que exista un trayecto activo. El servicio valida la relación del usuario o dispositivo con el estudiante, revisa el estado del trayecto y guarda latitud, longitud y hora de registro. La respuesta que recibe el cliente se genera a partir del modelo de transferencia; no corresponde a una escritura libre en la base. Además, el código contempla un filtro de desplazamientos improbables que compara distancias y tiempo transcurrido, para evitar registrar ciertos saltos inconsistentes. Ese filtro es una defensa heurística y debe calibrarse con ensayos, porque no reemplaza una evaluación formal de precisión geográfica.

Después de guardar una coordenada aceptada, el backend procesa posibles alertas y publica la actualización hacia el tópico STOMP del trayecto. El acudiente con una sesión conectada puede recibirla y modificar la vista del mapa. La secuencia puede describirse como «dispositivo — API — persistencia — publicación — cliente», pero no demuestra por sí sola una latencia determinada. Para medir tiempo real es necesario registrar marcas temporales consistentes en el dispositivo, el servidor y el receptor, considerando diferencias de reloj y pérdida de conexión.

El sistema distingue estados como `PENDING`, `IN_PROGRESS`, `COMPLETED` y `CANCELED`. Sin embargo, los nombres de un enum no prueban que todas las transiciones tengan una ruta pública de API o una pantalla disponible. En particular, el servicio revisado implementa una eliminación lógica: registra `deleted_at`, y si el viaje estaba activo cambia su estado a cancelado y guarda hora de cierre. La operación está protegida por permisos de lectura de ubicación o administración y por una validación adicional del derecho a gestionar al estudiante. La documentación del usuario debe evitar llamar a esa operación «borrado definitivo».

El tratamiento de coordenadas plantea una tensión entre consulta útil y conservación limitada. El repositorio de base de datos incluye una función que permite depurar coordenadas de trayectos terminados o cancelados anteriores a un período definido, pero la función no opera de forma automática. Es una herramienta para ejecutar una política institucional previamente aprobada, no evidencia de que dicha política exista. Para datos de ubicación de menores, la retención debe justificarse por finalidad y necesidad, no por la simple capacidad técnica de almacenar cada punto.

Disponibilidad de ubicación y manejo de desconexiones

Un mapa que conserva la última posición no informa necesariamente dónde se encuentra el estudiante en el momento presente. Para tratar ese problema, el backend incluye un servicio que revisa trayectos activos y determina si la última coordenada supera un intervalo de antigüedad configurado. El valor predeterminado de la revisión es de 120 segundos de inactividad y el proceso programado utiliza 30 000 milisegundos como intervalo de comprobación. Estas cifras son parámetros del código; no representan resultados de laboratorio ni umbrales clínicos o de seguridad (GPS Guardián Escolar, 2026a).

Al identificar una ubicación desactualizada, el servicio prepara un mensaje para usuarios asociados al estudiante y a sus acudientes activos. El texto señala que puede existir pérdida de internet o GPS o interrupción del envío de ubicación. Ese lenguaje es apropiado porque evita identificar como incidente una condición que tiene explicaciones técnicas alternativas. El sistema intenta generar el aviso sin duplicarlo dentro del período considerado, aunque su recepción en el teléfono del acudiente debe verificarse por separado.

Conviene distinguir tres estados operativos. El primero es «no hay posición todavía», que puede ocurrir al comenzar el recorrido. El segundo es «última posición recibida», útil si se muestra la hora. El tercero es «posición reciente», cuando el registro satisface una ventana temporal definida. La interfaz y los manuales deberían representar estas diferencias de forma explícita; de lo contrario, la persona puede interpretar el marcador como prueba de presencia en un lugar aun cuando su información sea antigua.

La continuidad también depende del sistema operativo móvil. Una tarea correctamente declarada puede detenerse por permisos denegados, políticas del fabricante, cierre de la aplicación o restricciones de batería. Para probarla se requiere un dispositivo físico, una compilación apropiada, permisos registrados y períodos de pantalla apagada. Expo señala requisitos específicos para estas pruebas y separa el comportamiento de una build de desarrollo del de Expo Go (Expo, s. f.-a). Por eso la capacidad implementada se considera una hipótesis operativa que necesita evidencia de campo controlada.

Mapas, zonas seguras y geocodificación

El módulo de mapas ayuda a convertir direcciones en coordenadas y a obtener información cartográfica asociada con rutas y ubicaciones. El proyecto eligió un servicio intermediario en Spring Boot para invocar funciones de Google Maps, de modo que las claves utilizadas por el servidor no deban incluirse en el cliente. El backend aplica mecanismos de caché, limitación de solicitudes y protección ante fallos de proveedores externos. Estas decisiones procuran controlar consumo, costos y exposición de credenciales (GPS Guardián Escolar, 2026a).

La geocodificación contiene incertidumbre que debe reconocerse. Una dirección puede ser ambigua, estar escrita de manera incompleta o corresponder a una zona donde el proveedor dispone de información limitada. Por eso el resultado cartográfico requiere revisión, especialmente al definir puntos sensibles como origen de ruta o domicilio. El mapa es una herramienta de apoyo para seleccionar y visualizar posiciones, no una certificación de la dirección ingresada.

Las zonas seguras permiten relacionar un estudiante con un área geográfica representada por centro y radio. El modelo incluye distancia en metros, nombre y parámetros que pueden intervenir en la generación de alertas. Su uso puede facilitar la identificación de proximidad a puntos importantes, pero un perímetro geográfico no es una barrera física: una medición imprecisa puede registrar una entrada o salida aparente sin que el estudiante haya cruzado realmente el límite. La validación necesita ensayos con distintas condiciones de señal y teléfonos.

La dependencia de un proveedor externo exige manejar fallos sin comprometer el resto del sistema. Cuando la API de mapas no responde, la gestión de estudiantes y el historial no deberían quedar inaccesibles por una excepción propagada de manera indiscriminada. Las medidas de resiliencia del módulo son una respuesta de diseño a ese riesgo, aunque su desempeño se debe comprobar con escenarios de cuota agotada, claves inválidas, latencia prolongada y desconexión. La ejecución de un test controlado sobre fallos externos permite verificar que los mensajes para el usuario sean claros y no revelen detalles internos de configuración.

Notificaciones y comunicación con acudientes

Las notificaciones conectan los eventos del dominio con la experiencia del acudiente. El repositorio contiene servicios para almacenar avisos, consultar una bandeja, marcar lecturas, gestionar preferencias y administrar dispositivos receptores. La existencia de un evento no equivale a que un mensaje haya llegado al destinatario: se requiere diferenciar la generación interna, el intento de envío, la aceptación del proveedor y la recepción visible. Esta distinción resulta especialmente importante en avisos vinculados a ubicación escolar (GPS Guardián Escolar, 2026a).

El backend contempla canales de correo y Expo Push según configuración. Para el correo, la arquitectura permite seleccionar SMTP o una API HTTPS de proveedor; para push, debe existir un token de dispositivo y estar habilitado el envío. En ambos casos hay dependencias externas: conectividad, cuotas, credenciales, autorización del remitente y políticas del teléfono receptor. El artículo puede describir esos puntos de integración, pero no puede afirmar tasas de entrega sin evidencias que vinculen cada evento con su resultado.

Un diseño cuidadoso necesita evitar alertas repetitivas que terminen por ser ignoradas. El servicio de ubicación desactualizada incorpora una operación orientada a generar una notificación única desde un momento de referencia. Se trata de un mecanismo útil, aunque su comportamiento requiere pruebas con eventos simultáneos, cambios de conexión y reinicios. La calidad de una notificación no depende solo de que se envíe; también de que su contenido, prioridad y frecuencia correspondan a una situación comprensible para la familia.

La interfaz de notificaciones debe permitir consultar el historial y distinguir mensajes nuevos de mensajes vistos. Además, las preferencias del usuario no deberían desactivar indiscriminadamente avisos que una institución considere obligatorios sin una decisión de negocio clara. El proyecto conserva infraestructura para opciones de notificación, pero se necesita un documento de políticas que defina qué avisos son informativos, cuáles son críticos y qué canales se usan en cada caso. Esa definición no se resuelve exclusivamente mediante código.

Modelo de datos y coherencia de las relaciones

PostgreSQL es el repositorio principal de entidades y eventos del sistema. La estructura incluye usuarios, roles, permisos, sesiones, estudiantes, acudientes, contactos de emergencia, rutas, paradas, asignaciones, trayectos, coordenadas, zonas seguras, notificaciones y registros de auditoría. El esquema separado evita mezclar información de distinta finalidad y permite definir restricciones que complementan las validaciones de los servicios (GPS Guardián Escolar, 2026c).

El proyecto utiliza identificadores UUID para relacionar recursos. Esta elección ayuda a evitar que el identificador de una entidad dependa de la cantidad de registros anteriores, pero no sustituye las comprobaciones de propiedad o autorización. Conocer el UUID de un estudiante no debe permitir consultar su información. El control efectivo ocurre cuando la API verifica que el

usuario autenticado dispone de una relación autorizada. Las restricciones de clave foránea, a su vez, protegen la coherencia entre registros dentro de una transacción.

Liquibase administra la evolución del esquema y mantiene el historial de changesets aplicados. El repositorio organiza cambios de DDL, DML y permisos en carpetas diferenciadas, con registros de rollback para operaciones específicas. La migración de ruta preferida constituye un ejemplo de cambio coordinado: antes de desplegar un backend que consulta `is_preferred`, la base necesita incorporar la columna y las restricciones correspondientes. Liquibase recomienda tratar los changesets aplicados como historial y elaborar cambios nuevos para ajustes posteriores (Liquibase, s. f.).

La eliminación lógica mantiene determinados registros con un campo de baja en lugar de retirarlos físicamente de inmediato. Esto protege trazabilidad y evita que la desaparición de un trayecto desarticule otras referencias. Sin embargo, la baja lógica también obliga a que las consultas filtren recursos inactivos de forma consistente. Un endpoint que ignore esa condición podría mostrar un trayecto que el usuario considera retirado; una búsqueda de ruta preferida que la ignore podría reutilizar datos inadecuados.

El backend emplea JPA e Hibernate con verificación del esquema en lugar de creación automática en producción. Esta configuración impide que la API corrija silenciosamente tablas durante el arranque. La responsabilidad de modificar estructuras queda en las migraciones; si falta una columna, el despliegue debe detenerse y hacer visible la inconsistencia. El enfoque incrementa el esfuerzo de coordinación entre repositorios, pero reduce cambios no registrados en la base.

Infraestructura local, Render y operación

El entorno local utiliza Docker Compose para definir redes, volúmenes, contenedores y dependencias. La base de datos PostgreSQL se inicia primero; Liquibase espera que esté saludable y aplica las migraciones; el backend se conecta con credenciales de aplicación y el frontend web queda detrás de Nginx. El diseño del Compose incluye comprobaciones de salud y expresa la finalización correcta de Liquibase como condición para iniciar servicios dependientes. La secuencia transforma instrucciones manuales dispersas en una configuración que puede versionarse (GPS Guardián Escolar, 2026d).

Para la publicación web, los antecedentes del proyecto describen un despliegue con servicios Docker en Render, una base PostgreSQL administrada y ajustes de variables de entorno. La elección de Render respondió a la conveniencia de disponer de una URL pública para la demostración sin administrar inicialmente un servidor completo. Sin embargo, la documentación del despliegue refleja una ejecución concreta y no equivale a un certificado de disponibilidad permanente. La naturaleza del plan contratado, los reinicios y las dependencias de correo pueden modificar el comportamiento sin que cambie el código.

Una lección relevante del despliegue es el orden de cambios. El esquema debe estar preparado antes de iniciar una versión de backend que lo utiliza. En la experiencia documentada, las migraciones de Liquibase se aplicaron a una base remota mediante un proceso externo al servicio web. La validación de arranque mostró conectividad con PostgreSQL y funcionamiento básico del servicio, aunque existieron advertencias y problemas de configuración de correo durante distintas etapas. Por eso resulta necesario registrar para cada despliegue el commit, la base utilizada, las variables no secretas y el resultado de salud.

El cliente web puede compilarse como sitio estático y publicarse mediante Nginx. El proxy dirige solicitudes `/api` al backend y conserva el canal `/ws` para WebSocket. Esta solución reduce dificultades de origen cruzado en el navegador, pero requiere una configuración correcta de HTTPS y encabezados de proxy. Un error en el host ascendente puede afectar todas las operaciones aun cuando el contenedor del frontend se encuentre saludable. La observación del contenedor y la validación funcional deben tratarse como dos comprobaciones diferentes.

La recuperación ante fallos no puede depender de volver a ejecutar un comando sin revisar consecuencias. Revertir la imagen de frontend o backend es diferente de revertir una migración de base de datos. Una migración puede alterar o eliminar información y requerir respaldo; un `docker compose down -v` eliminaría el volumen de datos del entorno. El repositorio de infraestructura advierte que el rollback necesita plan y versiones aprobadas. En una instalación que maneje datos de estudiantes, el procedimiento de recuperación debe ensayarse con información de prueba antes de ejecutarse sobre datos operativos.

Pruebas de software y criterios de aceptación

La prueba funcional más simple consiste en ejecutar una operación conocida y comprobar su respuesta. Sin embargo, un sistema de seguimiento necesita un conjunto coordinado de pruebas. Es posible que el registro funcione y el seguimiento falle; también puede ocurrir que el mapa reciba coordenadas mientras el control de acceso permita consultas indebidas. Por ello, una evaluación razonable debe separar pruebas de unidad, integración, contratos, interfaz, seguridad, despliegue y dispositivos físicos, sin declarar una etapa aprobada porque otra terminó correctamente.

Los repositorios incluyen archivos de pruebas del backend para autenticación, vínculos, acceso a estudiantes, rutas preferidas, trayectos y ubicación, además de pruebas de frontend para servicios HTTP, seguimiento y componentes. La existencia de estos archivos es evidencia del esfuerzo de automatización. No constituye, por sí sola, un resultado de ejecución vigente. Para reportar cobertura o éxito deben conservarse la orden utilizada, el commit exacto, el entorno, la fecha y el reporte generado por la herramienta. Esa disciplina evita citar cifras históricas de pruebas como si correspondieran a la versión actual.

En autenticación deben probarse el registro válido, entradas incompletas, correo no verificado, recuperación de contraseña, segundo factor, confirmación de un dispositivo nuevo, cierre de sesión y bloqueo ante intentos repetidos. El objetivo no es demostrar que una pantalla presenta un mensaje, sino verificar que la API rechaza estados no autorizados. Los casos negativos son decisivos: un token caducado no debe permitir operaciones, un dispositivo desvinculado no debe actualizar información y una cuenta sin permisos no debe alterar recursos de un estudiante ajeno.

Para seguimiento se requieren pruebas de inicio de trayecto, ausencia de ruta preferida, rutas múltiples sin selección, inserción de coordenadas, filtrado de saltos improbables, pérdida de conectividad, reanudación y consulta de historial. Debe registrarse la diferencia entre hora de captura y hora de recepción de cada punto. También conviene usar trayectos simulados para repetir eventos en condiciones controladas, y después realizar pruebas con dispositivos reales para detectar particularidades de permisos, ahorro de energía y redes móviles.

Las comprobaciones no funcionales incluyen tiempo de respuesta, estabilidad durante interrupciones, seguridad de los datos y consumo de recursos. ISO/IEC 25010:2023 puede orientar la selección de criterios; OWASP y NIST aportan referencias para preparar pruebas de

seguridad y desarrollo confiable (ISO, 2023; NIST, 2022; OWASP Foundation, s. f.). No se declara conformidad con estos modelos: se proponen para construir un protocolo que pueda auditarse.

La aceptación final necesita evidencias legibles por alguien distinto al desarrollador. Para un caso de prueba debería incluir identificador, objetivo, precondiciones, pasos, resultado esperado, resultado obtenido, fecha, versión y responsable. Las capturas de pantalla son útiles, pero en un endpoint también conviene conservar código de estado y fragmentos de respuesta sin datos personales. En móviles debe registrarse versión de Android, modelo, build, permisos y condiciones de batería. Este nivel de detalle permite investigar fallos sin depender únicamente del recuerdo del equipo.

Protección de datos personales y responsabilidad ética

La ubicación de un estudiante permite inferir desplazamientos, horarios y lugares frecuentados. Combinada con datos de contacto y fotografía, puede tener consecuencias relevantes si se comparte con personas no autorizadas. Por tanto, la seguridad del proyecto no debería examinarse únicamente como prevención de ataques informáticos. También exige definir quién recoge la información, para qué finalidad, por cuánto tiempo, en qué condiciones se comparte y cómo puede solicitarse su actualización o eliminación cuando corresponda.

En Colombia, la Ley 1581 de 2012 establece principios generales para el tratamiento de datos personales y reglas especiales respecto de información de niños, niñas y adolescentes. El Decreto 1377 de 2013 desarrolla aspectos sobre autorizaciones y tratamiento dentro del marco legal. La aplicación de estas normas a una operación concreta depende del contexto institucional y debe ser revisada por el responsable del tratamiento; el proyecto formativo no acredita por sí

solo esa conformidad jurídica (Congreso de Colombia, 2012; Presidencia de la República de Colombia, 2013).

En el diseño técnico, una primera barrera consiste en comprobar que quien consulta un recurso posee un vínculo legítimo con el estudiante. El backend implementa controles de rol, permisos y acceso por estudiante, así como validaciones específicas para dispositivos infantiles. La segunda barrera es reducir exposición: no incluir secretos del servidor en el cliente, evitar URLs con tokens persistentes y procurar que los mensajes de error no revelen datos de terceros. La tercera es la trazabilidad, que permite revisar cambios sin abrir indiscriminadamente toda la información personal.

La retención de coordenadas plantea una decisión que el equipo debe dejar por escrito. Conservar un historial indefinido puede parecer conveniente para investigar fallos, pero aumenta el volumen de información sensible acumulada. La base de datos ofrece una función manual de depuración de coordenadas de trayectos cerrados anteriores a cierta fecha; falta convertir esa capacidad en política aprobada, con plazos, criterios de excepción, responsable y procedimiento de respaldo. La función técnica es solo una parte de la solución.

También se necesita claridad para el acudiente y para el estudiante. El consentimiento no se reduce a marcar una casilla vacía o a mostrar un enlace extenso; la persona debe comprender qué datos se recogen y cómo puede retirar una vinculación. La interfaz de registro y las pantallas de permisos deben comunicar límites reales del seguimiento. Una solución que promete ubicación «siempre disponible» sin poder sostenerla técnicamente no ofrece información transparente, aun cuando su intención sea tranquilizar a las familias.

Resultados de la revisión documental y del código

El resultado principal es la identificación de una estructura técnica consistente con los objetivos funcionales del proyecto. Se observaron módulos de autenticación, estudiantes, rutas, trayectos, zonas, mapas, notificaciones y administración en el backend; pantallas y servicios relacionados en el frontend; un esquema de PostgreSQL con migraciones; y un Compose que establece dependencias de arranque. Estas observaciones describen componentes presentes y permiten reconstruir cómo está concebida su interacción, pero no significan que todas las funciones hayan sido verificadas mediante pruebas de extremo a extremo en la fecha del corte (GPS Guardián Escolar, 2026a, 2026b, 2026c, 2026d).

En la consulta del código de `main` se identificó una regla concreta de selección de rutas: preferencia activa, selección única como alternativa y error ante varias rutas sin criterio. También se observó el inicio de un trayecto en estado `IN_PROGRESS`, la recepción de coordenadas, la publicación a un tópico STOMP y el borrado lógico con cancelación de un recorrido activo. Son resultados de inspección estática porque se fundamentan en métodos y condiciones del repositorio. No se asume a partir de ellos que el circuito completo haya funcionado en Android durante un recorrido real.

El servicio de disponibilidad de ubicación representa otro hallazgo. Su tarea programada compara la última coordenada con una ventana temporal y prepara un aviso cuando la información está desactualizada. Esta función responde a un problema real del seguimiento: presentar una posición antigua como si fuera actual. Aun así, para evaluar si la alerta llega oportunamente debe realizarse un ensayo controlado en el que se interrumpa el envío, se mida el instante del aviso y se verifique el canal receptor.

El expediente de despliegue informa que se realizó una puesta en marcha en Render, se aplicaron migraciones a PostgreSQL y se observaron comprobaciones de conectividad y arranque del backend. Esa información es un antecedente operativo valioso, no una supervisión continua. No se dispone en este artículo de un conjunto de registros actualizado que permita calcular disponibilidad mensual, capacidad de concurrencia o tiempos medios de recuperación. La conclusión se limita a que existe una ruta documentada de implementación y evidencias parciales de ejecución.

La inspección detectó asimismo un problema de consistencia documental: algunos documentos anteriores describían una modificación de estado de trayecto mediante `PATCH`, mientras que el controlador de trayectos de `main` examinado no lo exponía. Este hallazgo no demuestra automáticamente que el proyecto carezca de toda operación de finalización, porque podría existir otro flujo o una diferencia entre ramas; sí obliga a que los manuales y las pruebas no atribuyan a ese controlador una ruta que no aparece en su implementación revisada. La trazabilidad entre código y documento debe actualizarse cada vez que cambia el contrato.

Por último, se identificó una brecha entre las pruebas que el proyecto necesita y los reportes que están efectivamente adjuntos. Hay clases de pruebas y comandos de calidad descritos, pero no se aporta una campaña completa y reproducible de seguimiento Android, lectura QR en varios modelos, recepción de notificaciones y precisión de ubicación con datos medidos. El resultado no es «fracaso» ni «éxito» de esas funciones: es una evaluación incompleta que debe consignarse como tal hasta obtener evidencia suficiente.

Discusión: fortalezas y tensiones de la solución

La fortaleza más visible de GPS Guardián Escolar es que sus componentes no fueron diseñados como demostraciones aisladas. El registro de estudiantes se relaciona con el acceso de

acudientes; la asignación de rutas interviene en la creación de trayectos; las coordenadas se almacenan y también se publican para consulta; las notificaciones aprovechan eventos del seguimiento. Esa continuidad entre funciones permite estudiar una aplicación de alcance formativo relativamente amplio y reconocer puntos de integración donde los defectos tienen consecuencias en distintas capas.

La misma integración trae complejidad. Un cambio de contrato en backend puede exigir modificar servicios y pantallas del frontend; una columna nueva debe existir antes de desplegar el servicio que la consulta; una política de autorización tiene que aplicarse tanto al canal HTTP como al de tiempo real. Las diferencias detectadas entre ramas de desarrollo, QA y publicación muestran que no es suficiente decir que un repositorio está «actualizado». Es necesario señalar cuál commit representa cada entorno y cuándo una modificación se encuentra realmente integrada.

La seguridad plantea una tensión entre comodidad y control. La vinculación QR reduce pasos para asociar teléfonos, pero facilita compartir un enlace por error. Los mecanismos de confirmación y caducidad introducen pasos adicionales, aunque resultan pertinentes al tratarse de menores. Del mismo modo, ocultar acciones de administración en la interfaz infantil mejora la experiencia, pero no sustituye la autorización del backend. El diseño debe asumir que una solicitud puede ser construida fuera de la aplicación oficial.

El tiempo real también exige moderar expectativas. Publicar coordenadas por WebSocket puede producir una experiencia de seguimiento fluida cuando la red está disponible; no garantiza cobertura constante ni precisión uniforme en interiores, zonas densas o teléfonos con restricciones energéticas. Por ello, una buena interfaz no debería disimular la antigüedad del último dato. El servicio que detecta ubicación obsoleta representa un avance relevante, y al

mismo tiempo demuestra por qué un sistema confiable necesita comunicar incertidumbre al usuario.

Desde la perspectiva del mantenimiento, la separación de repositorios y el uso de migraciones son decisiones defendibles, pero obligan a documentar una secuencia de release. La solución no se beneficia de fusionar todas las ramas sin revisión. Una versión de referencia, integrada y acompañada de pruebas, ofrece una base más sólida para redactar manuales, hacer presentaciones y evaluar defectos. En un proyecto con varios colaboradores, la calidad de la coordinación termina siendo parte de la calidad del software.

Limitaciones del estudio

El análisis se sustenta en documentos y código, no en un experimento con participantes. No se incluyeron familias usuarias, operadores de transporte o una muestra de dispositivos que permitiera comparar precisión de localización, satisfacción o percepción de seguridad. Por esa razón no se reportan porcentajes de reducción de riesgos, variaciones de ansiedad, desempeño estadístico ni mejoras sociales atribuibles al proyecto. Hacerlo requeriría otro diseño metodológico y autorización para tratar datos de personas.

Tampoco se cuenta con una prueba operativa continua del sistema publicada junto con los artefactos consultados. Un servicio que respondió durante una demostración pudo tener un comportamiento diferente después de cambios en variables, credenciales o cuotas. La revisión del repositorio permite comprobar la presencia de rutas y configuraciones; la verificación productiva necesitaría monitoreo y registros con hora, versión y entorno. Las conclusiones se formulan deliberadamente dentro de ese límite.

El proyecto sigue sujeto a proveedores externos y plataformas móviles. La API de mapas, el correo y las notificaciones dependen de servicios que pueden variar en precio, disponibilidad,

requisitos de autenticación y límites. Android introduce restricciones específicas de ubicación en segundo plano. Estos factores no se controlan únicamente desde el código fuente y justifican que las pruebas se repitan antes de una entrega institucional.

Una limitación adicional es documental. Las revisiones sucesivas conservaron descripciones de distintas etapas; algunas se anticiparon al comportamiento posterior del código. La comparación de ramas y la auditoría de controladores permitieron detectar una parte de las diferencias, pero no equivalen a una revisión exhaustiva de todas las HU históricas. El presente artículo selecciona flujos centrales y orienta el proceso para mantener la correspondencia entre requisitos, implementación y evidencia de aceptación.

Plan de evaluación y trabajo futuro

La primera etapa recomendada es definir un corte de software con commit completo de backend, frontend, base de datos e infraestructura, acompañado de una base de datos de prueba anonimizada o sintética. Con ese corte se debe ejecutar el conjunto de pruebas automatizadas y guardar los reportes de compilación, lint, unidad e integración. En el reporte se indicará qué falló, qué no pudo ejecutarse y qué dependencia impidió continuar. No conviene convertir una prueba omitida en un resultado favorable.

La segunda etapa debe validar seguridad y autorización. Se proponen cuentas de prueba con distintos roles, estudiantes compartidos, dispositivos vinculados y desvinculados y sesiones concurrentes. Cada caso debe documentar la solicitud y el resultado esperado, evitando exponer claves, tokens o datos personales en las capturas. Para los flujos de recuperación y segundo factor se requieren cuentas de correo de prueba configuradas y evidencia de rechazo cuando no se satisfacen las condiciones. Los controles recomendados pueden contrastarse con OWASP y NIST (NIST, 2022; OWASP Foundation, s. f.).

La tercera etapa consiste en ensayar localización en Android físico. Deben registrarse modelo, sistema operativo, tipo de compilación, estado de batería, permisos y red; después se ejecutarán trayectos controlados con pantalla encendida, pantalla apagada y cambios de cobertura. La evaluación deberá medir número de coordenadas recibidas, intervalos entre envíos, precisión aproximada frente a puntos de referencia y tiempo de aviso por ubicación desactualizada. Se recomienda no utilizar trayectos reales de menores para esta primera campaña.

La cuarta etapa corresponde al despliegue reproducible. El equipo debería ensayar restauración de respaldo, aplicación de migraciones, inicio ordenado del Compose y recuperación a una versión anterior. También debe comprobar el comportamiento de Nginx, WebSocket y los proveedores externos bajo errores provocados. Una instalación queda mejor caracterizada cuando se conoce su respuesta ante fallos que cuando solo se demuestra el arranque en condiciones favorables.

La quinta etapa es de validación de experiencia y gobernanza. Antes de usar datos reales, una institución tendría que revisar los textos de consentimiento y privacidad, establecer una política de retención, designar responsables de atención de solicitudes y probar el recorrido completo con personas adultas que representen las tareas del acudiente. Deberían evaluarse claridad de mensajes, comprensión de estados de ubicación y facilidad para desvincular un teléfono. Esas actividades completarían la mirada técnica con una verificación del uso previsto.

Conclusiones

GPS Guardián Escolar materializa un proyecto formativo que articula requisitos, interfaz, servicios, persistencia e infraestructura alrededor de una necesidad concreta: acompañar recorridos escolares con información de ruta y ubicación asociada a estudiantes y acudientes autorizados. Los repositorios permiten reconocer una implementación modular, contratos REST, un canal de publicación STOMP, migraciones Liquibase y un entorno Docker. También muestran decisiones relevantes sobre autenticación, vínculos de dispositivo y control de acceso.

El caso confirma que el núcleo de un sistema de seguimiento no reside solamente en obtener coordenadas. Se requiere relacionarlas con un trayecto vigente, impedir accesos fuera de las relaciones permitidas, ofrecer información sobre la antigüedad del dato y conservar trazabilidad. La selección de ruta preferida, la cancelación lógica y el aviso por ubicación desactualizada ilustran decisiones que cruzan varias capas y que deben mantenerse coherentes entre código, base de datos, cliente y manuales.

Los resultados disponibles son principalmente de análisis técnico y de ejecución parcial documentada. No permiten afirmar que el producto haya demostrado precisión GPS en un conjunto de dispositivos, disponibilidad continua, entrega garantizada de avisos ni efecto comprobado sobre la seguridad de los trayectos. Reconocer estos límites fortalece el valor del trabajo porque orienta pruebas medibles en lugar de atribuir resultados no observados.

La continuidad del proyecto debe centrarse en estabilizar un corte de release, completar las verificaciones de extremo a extremo, actualizar los documentos según el código aprobado y formalizar políticas de protección de información de menores. Con esas actividades, GPS Guardián Escolar podrá pasar de una solución técnicamente articulada a un producto cuya operación pueda evaluarse y discutirse con criterios claros.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Expo. (s. f.-a). *Location*. Expo documentation, SDK 57. https://docs.expo.dev/versions/v57.0.0/sdk/location/

Expo. (s. f.-b). *TaskManager*. Expo documentation. https://docs.expo.dev/versions/latest/sdk/task-manager/

GPS Guardián Escolar. (2026a). *Backend-Guardian-Escolar* [Repositorio de código fuente]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar

GPS Guardián Escolar. (2026b). *Fronted-GuardianEscolar-* [Repositorio de código fuente]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-

GPS Guardián Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Repositorio de código fuente]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar

GPS Guardián Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Repositorio de código fuente]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar

GPS Guardián Escolar. (2026e). *Docs-Guardian-Escolar* [Repositorio de documentación]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar

International Organization for Standardization. (2023). *ISO/IEC 25010:2023: Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model*. https://www.iso.org/standard/78176.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Liquibase. (s. f.). *How to handle changing Liquibase changesets*. https://www.liquibase.com/blog/dealing-with-changing-changesets

National Institute of Standards and Technology. (2022). *Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST SP 800-218). https://doi.org/10.6028/NIST.SP.800-218

OWASP Foundation. (s. f.). *Mobile Application Security Verification Standard (MASVS)*. https://mas.owasp.org/MASVS/

Presidencia de la República de Colombia. (2013). *Decreto 1377 de 2013: Por el cual se reglamenta parcialmente la Ley 1581 de 2012*. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=53646

Spring. (s. f.-a). *WebSockets: Spring Boot 3.5 reference*. https://docs.spring.io/spring-boot/3.5/reference/messaging/websockets.html

Spring. (s. f.-b). *Spring Security: Spring Boot 3.5 reference*. https://docs.spring.io/spring-boot/3.5/reference/web/spring-security.html

Apéndice A. Relación entre problemas, decisiones y evidencias

La matriz presentada a continuación organiza el núcleo del estudio en términos de necesidad, solución y estado de validación. No representa una auditoría de conformidad ni sustituye la especificación institucional de requisitos. Su finalidad es mostrar por qué determinadas decisiones se tomaron y qué tendría que comprobarse para aceptarlas.

Tabla 1

Relación entre necesidades, decisiones y evidencias

Necesidad	Implementación identificada	Evidencia y límite
Identificar al acudiente	Registro, correo, JWT, sesiones y 2FA	Código presente; validar flujos con proveedor activo
Asociar teléfono con estudiante	Código/QR, vínculo y desvinculación	Controladores y servicios; faltan ensayos por dispositivo
Elegir ruta de un trayecto	Asignación activa y ruta preferida	Servicio y migración 014; pruebas integradas pendientes
Consultar ubicación	Coordenadas y STOMP por trayecto	Código de publicación; latencia real no medida
Informar falta de señal reciente	Tarea con ventana de 120 s y revisión de 30 s	Parámetros del código; recepción externa pendiente
Conservar historial	Tablas, paginación y baja lógica	Persistencia descrita; retención institucional por definir
Proteger datos personales	Roles, permisos y relaciones por estudiante	Requiere pruebas negativas y política de tratamiento
Implementar en entorno reproducible	Docker, Liquibase y health checks	Guías presentes; recuperación y operación a certificar

Nota. El estado de evidencia se refiere al corte documental; no es certificación funcional.

Apéndice B. Especificación de pruebas pendientes

Los casos constituyen una propuesta de ejecución para la versión de referencia; no son resultados aprobados. Cada registro deberá contener evidencia sin información sensible y asociarse al commit examinado.

Tabla 2

Casos propuestos para pruebas de aceptación

Caso	Procedimiento básico	Criterio de aprobación
PR-01	Registrar cuenta y confirmar correo	Usuario entra solo tras validaciones previstas
PR-02	Solicitar acceso desde dispositivo no reconocido	Flujo de confirmación requerido; rechazo controlado
PR-03	Vincular, usar y desvincular teléfono infantil	Después de desvincular, se rechazan operaciones protegidas
PR-04	Crear trayecto con una sola ruta y sin ruta preferida	Se selecciona la única ruta activa válida
PR-05	Crear trayecto con varias rutas sin seleccionar	Se solicita decisión explícita; no se elige arbitrariamente
PR-06	Publicar coordenadas durante trayecto activo	Se persisten y difunden para el suscriptor autorizado
PR-07	Detener envío por más de umbral	Se registra aviso sin atribuir falsamente incidente
PR-08	Intentar acceso a estudiante ajeno	Backend rechaza consulta y modificación
PR-09	Cancelar lógicamente viaje activo	Se conserva registro con estado cancelado y hora final
PR-10	Reiniciar sistema con esquema actualizado	Migraciones concluyen y servicios superan chequeos
PR-11	Desconectar proveedor Maps o agotar cuota	Error controlado; sin exposición de credenciales
PR-12	Probar app instalada con pantalla apagada	Evidencia GPS, permisos, dispositivo y tiempos observados

Nota. Todos los escenarios se presentan como pruebas por ejecutar; no se atribuyen resultados.

Apéndice C. Inventario sintético de contratos verificados

El cuadro recoge rutas relevantes encontradas en controladores del backend `main` en la revisión del 5 de noviembre de 2026. Su propósito es ilustrar el alcance del código observado; el catálogo OpenAPI publicado, si está disponible, debe utilizarse para la validación integral de parámetros y respuestas.

Tabla 3

Contratos representativos de la API

Dominio	**Operaciones representativas**	**Observación**
Autenticación	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me	Incluye correo, recuperación y 2FA
Estudiantes	GET y POST /api/students; DELETE /api/students/{studentId}	Listado paginado y permisos específicos
Rutas preferidas	PUT /api/students/{studentId}/preferred-route/{routeId}	Persistencia depende de la migración 014
Vinculación	POST /api/students/{studentId}/link-code; POST /api/students/link-device	Requiere validar vínculo y dispositivo
Trayectos	GET /api/trips; POST /api/trips; DELETE /api/trips/{tripId}	DELETE realiza baja lógica según servicio
Coordenadas	POST y GET /api/trips/{tripId}/coordinates	Consulta paginada y publicación STOMP
Notificaciones	GET /api/notifications; PATCH /api/notifications/{receiptId}/read	Preferencias y recepción dependen del entorno
Seguimiento en vivo	/ws y /topic/trips/{tripId}/coordinates	Control de suscripción debe verificarse

Nota. Las rutas se contrastaron con el código del backend en la rama main; verifique el contrato vigente antes de usarlo.

Apéndice D. Notas sobre reproducibilidad y límites de la evidencia

La revisión documental del 5 de noviembre de 2026 identificó como referencias de publicación los siguientes commits: Backend `1022058d545a`, Frontend `3d9f59fa3e91`, estructura de base de datos `9278b1dcb224` e infraestructura `131e98954430`. Para una reproducción rigurosa deben comprobarse los SHA completos en GitHub, la coherencia del esquema y el conjunto de variables del entorno sin incluir secretos en los anexos. Los hashes se utilizan como identificadores de inspección, no como garantía de que el sistema haya superado todas las pruebas.

Además de estas referencias, se conocían ramas de desarrollo con commits posteriores. Mezclar archivos de una rama de desarrollo con otras ramas de publicación podría producir una configuración que nunca estuvo desplegada de ese modo. Para el propósito académico se adopta el criterio de un corte coherente y se informa expresamente la naturaleza estática de la inspección. Cualquier revisión futura debe indicar nuevos commits, resultados CI y diferencias funcionales observadas.

Los ejemplos de prueba utilizan usuarios y recorridos sintéticos. Toda evaluación con personas o desplazamientos reales requiere autorización institucional y criterios adecuados de protección de datos personales.